

SEO AUTOMATION PLATFORM

End-to-End Product Documentation

From Technical SEO Automation to a Full AI-Assisted SEO Growth Platform

V1 target

A production-ready SaaS platform that connects GitHub, Google Search Console, GA4 and Google Business Profile; crawls websites; runs technical SEO and Lighthouse audits; detects issues; prepares safe fixes; creates GitHub pull requests; requires approval; and verifies the result after deployment.

Architecture principle: deterministic code is the source of truth for technical measurements. LLMs are optional and should be added only for language reasoning, recommendations and complex generation.

This document combines the current product concept with the previously prepared local-business operating blueprint.

Table of Contents

#	Section
1	Product vision and scope
2	Current product: what is already built
3	Technical SEO vs broader SEO
4	V1 → final-product evolution
5	End-to-end customer journey
6	System architecture
7	Technology stack
8	Frontend architecture
9	Backend, workers and queues
10	Crawler and headless-browser system
11	Technical SEO audit engine
12	GitHub remediation and approval gate
13	GSC, GA4 and Business Profile
14	Lighthouse and performance
15	Database and data model
16	Monitoring and change detection
17	Optional LLM / AI architecture
18	Security, testing and production
19	Deployment and DevOps
20	Final product modules
21	Implementation roadmap
22	Final production checklist

1. Product Vision and Scope

The platform is a multi-tenant SEO automation system designed to move beyond static SEO reports. It connects website source code, search data, analytics, local-business data and browser-based performance measurements into one workflow.

The central product loop is: **Connect → Crawl → Measure → Diagnose → Recommend → Approve → Change → Deploy → Verify → Measure Again.**

Primary V1 outcome

- Connect a website project to GitHub and supported Google services.
- Understand the website through crawling and browser rendering.
- Run deterministic technical SEO checks.
- Run Lighthouse/performance audits.
- Display GSC, GA4 and Business Profile data in a unified dashboard.
- Create safe change proposals for fixable issues.
- Create a GitHub branch and pull request.
- Require human approval before production-impacting changes.
- Re-crawl after deployment and verify whether the issue is resolved.
- Store historical snapshots so regressions can be detected later.

Long-term outcome

Expand from technical SEO into content SEO, keyword tracking, local SEO, competitor gap analysis, continuous monitoring, AI-assisted recommendations, agency workflows and eventually a continuous SEO engineering agent.

The supplied local-business plan uses the same philosophy: first validate the service, then automate repetitive work, with the technology serving the business process rather than becoming the process. The plan explicitly recommends Client #1 before heavy automation.

■filecite■turn0file0■L10-L14■

2. Current Product: What Is Already Built

Based on the current product description, the foundation already covers a meaningful part of the final architecture.

Existing capability	Role in product
GitHub connection	Source-code access and remediation
robots.txt detection	Technical SEO audit + automation
sitemap.xml detection	Technical SEO audit + automation
Create/update files	Deterministic remediation
GitHub PR creation	Safe review workflow
User consent before merge	Approval gate
GSC integration	Search performance and visibility
GA4 integration	Traffic and engagement
Business Profile integration	Local SEO/visibility
Lighthouse	Performance and technical quality
Headless Chrome	Real browser rendering and audit execution

This means you are not starting from zero. You already have the core pattern required for a serious V1: **observe** → **find** → **propose** → **review** → **execute**.

What is still missing for a complete platform is depth: more technical rules, continuous monitoring, richer analytics, broader SEO, stronger verification and eventually AI-assisted reasoning.

3. Technical SEO vs Broader SEO

Broader SEO	Technical SEO
Keyword research	Crawlability
Search intent	Indexability
Content strategy	Robots.txt
Content optimization	XML sitemap
Topic clusters	Canonical URLs
Backlink/authority work	Metadata
CTR optimization	Structured data
Content freshness	Redirects and HTTP status
Local content	Internal links and orphan pages

Current status

Your present platform is best described as **Technical SEO Automation + SEO Analytics + GitHub Remediation + Performance Auditing**. It is not yet the full scope of broader SEO.

That is not a problem. Technical SEO is a strong foundation because many checks are deterministic and can safely be automated. Broader SEO can be added as separate modules later.

Technical SEO expansion targets

- Redirect chains and loops.
- Broken links and orphan pages.
- Canonical conflicts and invalid canonical targets.
- Duplicate/missing metadata and heading problems.
- Structured-data extraction and validation.
- Image SEO and image-performance checks.
- JavaScript SEO: raw HTML versus rendered HTML.
- Mobile rendering and usability checks.
- Internationalization/hreflang where relevant.
- Duplicate/near-duplicate page signals.
- Internal-link depth and site architecture.

4. V1 → Final-Product Evolution

Stage	Build	Result
V1	Technical SEO + GitHub + GSC + GA4 + GBP + Lighthouse	Working technical SEO SaaS
V2	Deeper crawl, schema, links, images, JS SEO	Comprehensive technical SEO
V3	Snapshots, alerts, regression detection	Continuous SEO monitoring
V4	Keywords, content gaps, internal linking, local SEO	Broader SEO platform
V5	AI recommendations and SEO Copilot	AI-assisted SEO
V6	AI-assisted code/content generation + validation	Semi-automated execution
V7	Teams, billing, agency/white-label	Commercial SaaS
V8	Detect → fix → verify → measure loop	Continuous SEO operations

Do not build V8 before V1 is stable. Each version should be independently useful.

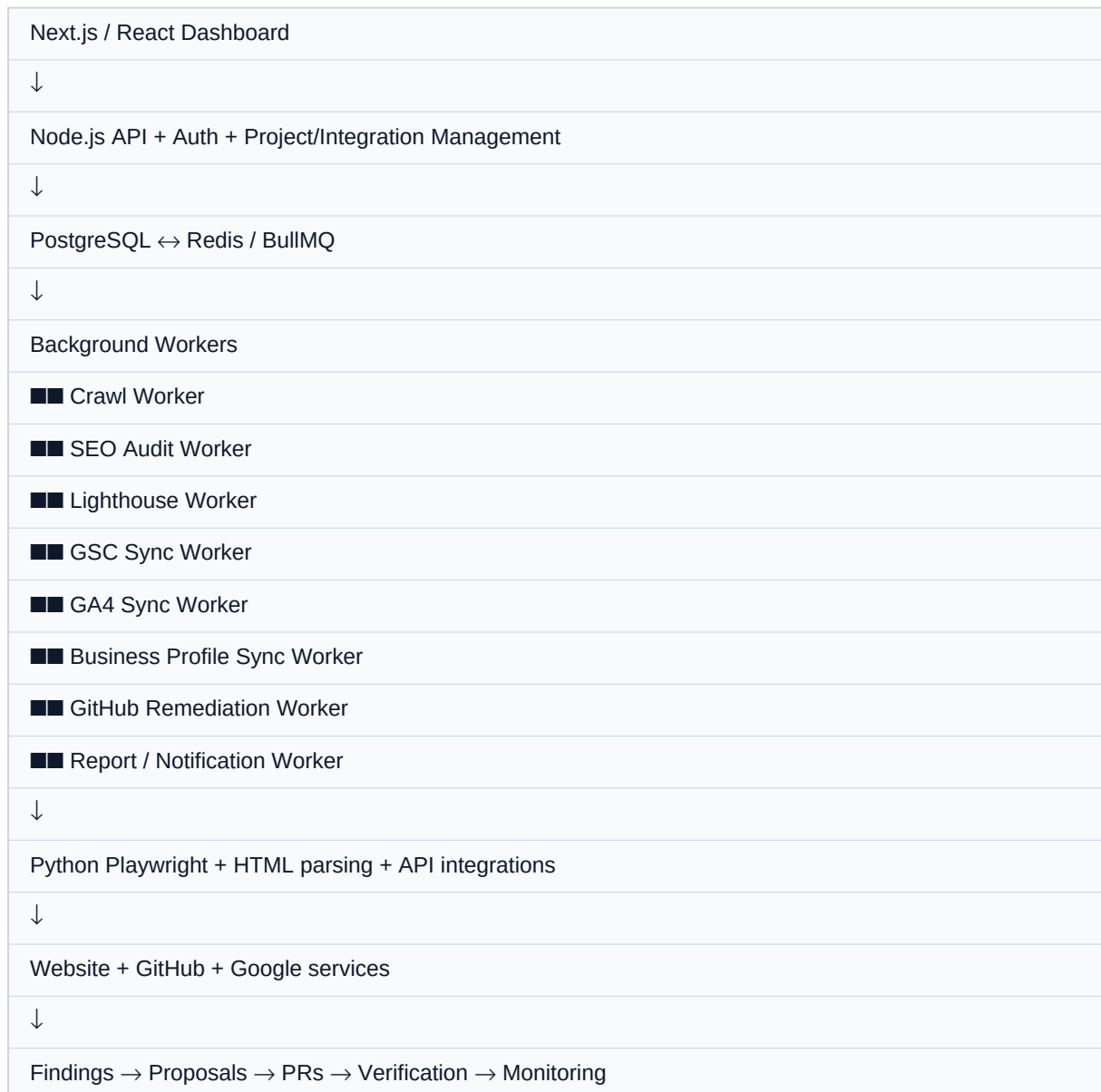
The previously prepared operating plan follows the same staged approach: manual validation, then CRM/crawler, then scoring, AI drafting, workflow automation and team scaling. ■filecite■turn0file0■L491-L506■

5. End-to-End Customer Journey

1. Sign up / create organization
↓
2. Create Website Project
↓
3. Connect GitHub repository
↓
4. Connect GSC + GA4 + Business Profile
↓
5. Discover production URL and framework
↓
6. Initial crawl + browser rendering + Lighthouse
↓
7. Technical SEO engine generates findings
↓
8. Dashboard groups issues by category/severity
↓
9. User opens an issue and sees evidence + recommendation
↓
10. Safe issue → Change Proposal
↓
11. GitHub branch + commit + PR
↓
12. User reviews diff and approves/rejects
↓
13. Merge + CI/CD deployment
↓
14. Post-deployment targeted crawl
↓
15. Finding verified / remains open
↓
16. Monitoring continues and metrics are compared over time

The important product experience is that the user should never have to understand the internal worker architecture. They should see a clear chain from **problem** → **evidence** → **proposed fix** → **approval** → **result**.

6. System Architecture



Service boundaries

Service	Responsibility
Frontend	Dashboard, onboarding, audit views, approvals
API	Authentication, CRUD, orchestration, integrations
Crawler	Website inspection and page inventory
SEO Engine	Deterministic rules and findings
Performance	Lighthouse execution and normalization
Google Integration	GSC, GA4, GBP synchronization
GitHub Integration	Branches, commits, PRs, checks
Workers	Long-running and scheduled jobs
AI Service	Optional language reasoning/generation
Storage	Reports, snapshots and artifacts

7. Technology Stack

Layer	Technology	Purpose
Web	Next.js + React	SaaS dashboard
UI	Tailwind CSS + component library	Responsive product UI
API	Node.js + Express	Business logic and integrations
DB	PostgreSQL	Application state and metrics
ORM	Prisma	Schema and migrations
Queue	Redis + BullMQ	Background jobs/schedules
Crawler	Python + Playwright	Browser-based crawling
Parsing	BeautifulSoup / lxml / requests	HTML/XML extraction
Performance	Lighthouse	Lab performance and SEO checks
Source control	GitHub API	Repository and PR automation
Search	Google Search Console API	Search performance
Analytics	GA4 Data API	Traffic/engagement
Local	Google Business Profile APIs where available	Local presence
Storage	AWS S3 or compatible	Artifacts and reports
Containers	Docker / Compose	Reproducible environments
Deploy	Vercel + Render/Railway/AWS	Production infrastructure
Observability	Structured logs + error tracking	Reliability
AI optional	Ollama/open model → paid LLM API	Reasoning/generation

This stack matches the supplied operating blueprint, which recommends Next.js/React, Node/Express, PostgreSQL, Redis/BullMQ, Python/Playwright, Lighthouse, S3-compatible storage and Docker, with local/open AI first and paid APIs later.

filecite turn0file0 L403-L443

8. Frontend Architecture

Organize the UI around Projects/Websites. Integrations are capabilities of a project, not the primary navigation structure.

```
app/ (auth)/ login/ register/ dashboard/ overview/ projects/ projects/[projectId]/ overview/  
technical-seo/ crawl/ performance/ search-console/ analytics/ business-profile/ issues/ changes/  
pull-requests/ reports/ monitoring/ settings/ onboarding/ github/ google/
```

Core screens

- Overview: health, trend, open issues, recent changes.
- Technical SEO: issue categories, filters and affected URLs.
- Issue detail: evidence, severity, recommendation and fixability.
- Change Proposal: files, before/after, validation and approval.
- PR Tracker: branch, checks, review and merge state.
- Performance: Lighthouse and Core Web Vitals trends.
- GSC/GA4/GBP: business-oriented analytics.
- Monitoring: regressions, alerts and scheduled jobs.
- Reports: exportable client-ready summaries.
- Settings: integrations, permissions, crawl limits and schedules.

UX principle

Do not expose raw technical data without interpretation. Every issue should answer: **What happened? Where? Why does it matter? What can be changed? How risky is the change?**

9. Backend, Workers and Queues

Full crawls and browser automation are long-running jobs. They belong in workers, not in synchronous API requests.

Queue	Representative jobs
crawl	crawl_site, crawl_url, recrawl_changed_pages
audit	run_rules, aggregate_findings
lighthouse	run_lighthouse, compare_metrics
gsc	sync_search_console, refresh_dimensions
ga4	sync_ga4, refresh_landing_pages
gbp	sync_business_profile
github	inspect_repo, create_branch, commit, create_pr
verify	post_deploy_verification
report	generate_pdf, generate_csv
notification	email/in-app alerts

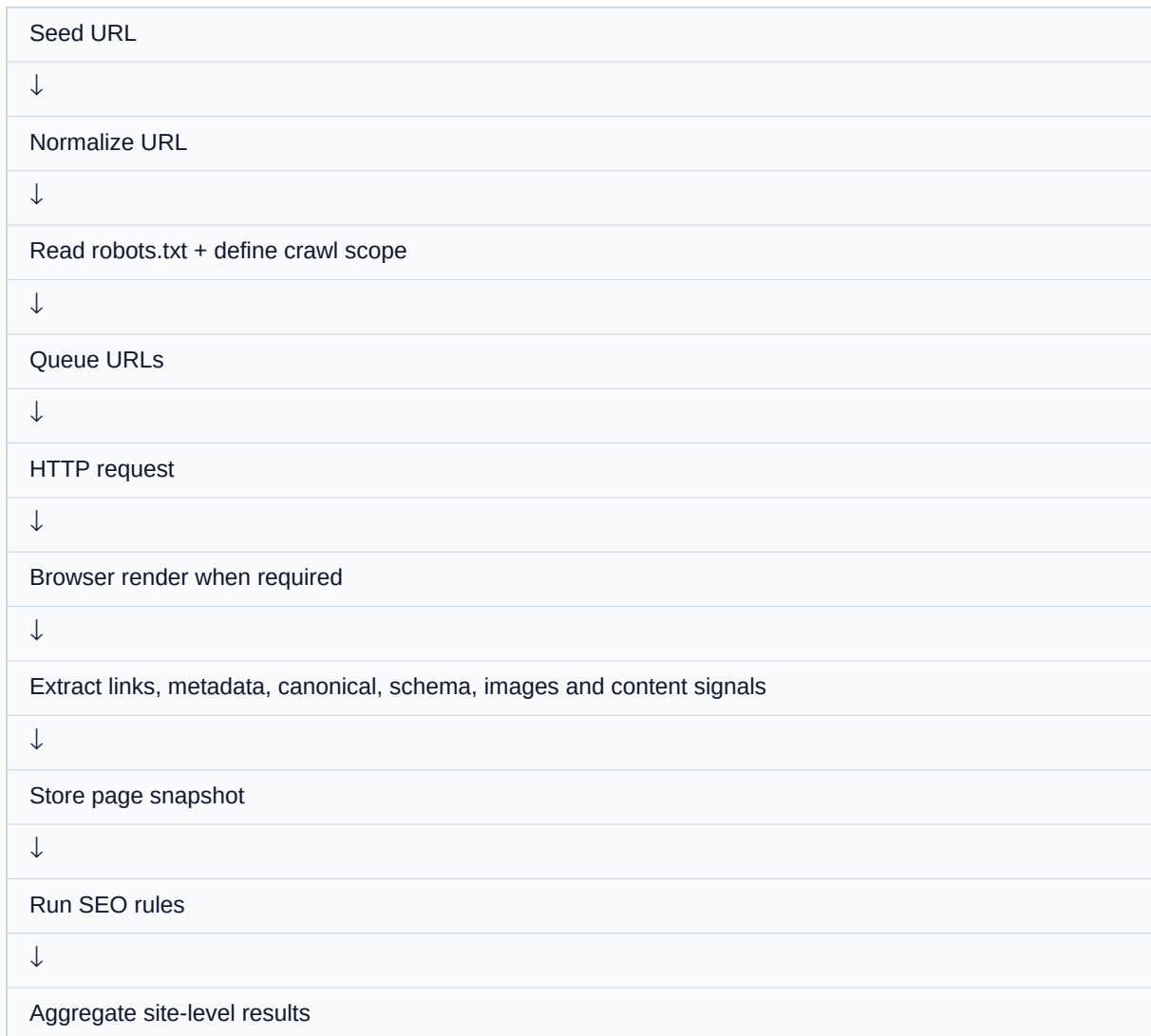
Example job state

QUEUED → RUNNING → SUCCESS ■ RETRY ■ FAILED

Every job should have a request/job ID, timestamps, retry policy, error reason and source project. This makes debugging and billing/usage measurement possible later.

10. Crawler and Headless-Browser System

Playwright/headless Chrome is especially useful because modern websites often render content through JavaScript. Your crawler should support both raw HTTP inspection and rendered-page inspection.



Crawler controls

- Maximum pages per project/plan.
- Configurable concurrency.
- Request timeout and retry policy.
- URL normalization and deduplication.
- Redirect tracking.
- Content-type filtering.
- Robots-aware crawling.
- Caching where appropriate.
- Crawl pause/cancel support.
- Respect website access restrictions and applicable terms.

Advanced JavaScript SEO

Compare raw response HTML against rendered DOM. Detect content, links, title, canonical and structured data that appear only after JavaScript execution. This becomes a differentiating feature for React/Next.js/Vue sites.

11. Technical SEO Audit Engine

The audit engine should be deterministic. Each rule returns a finding with a rule ID, category, severity, affected URL, evidence and recommended action.

```
{ "rule_id": "CANONICAL_TARGET_404", "category": "canonical", "severity": "high", "url":  
  "/services/seo", "evidence": { "canonical": "/services/old-seo", "status": 404 }, "fixable": true }
```

V1 rules

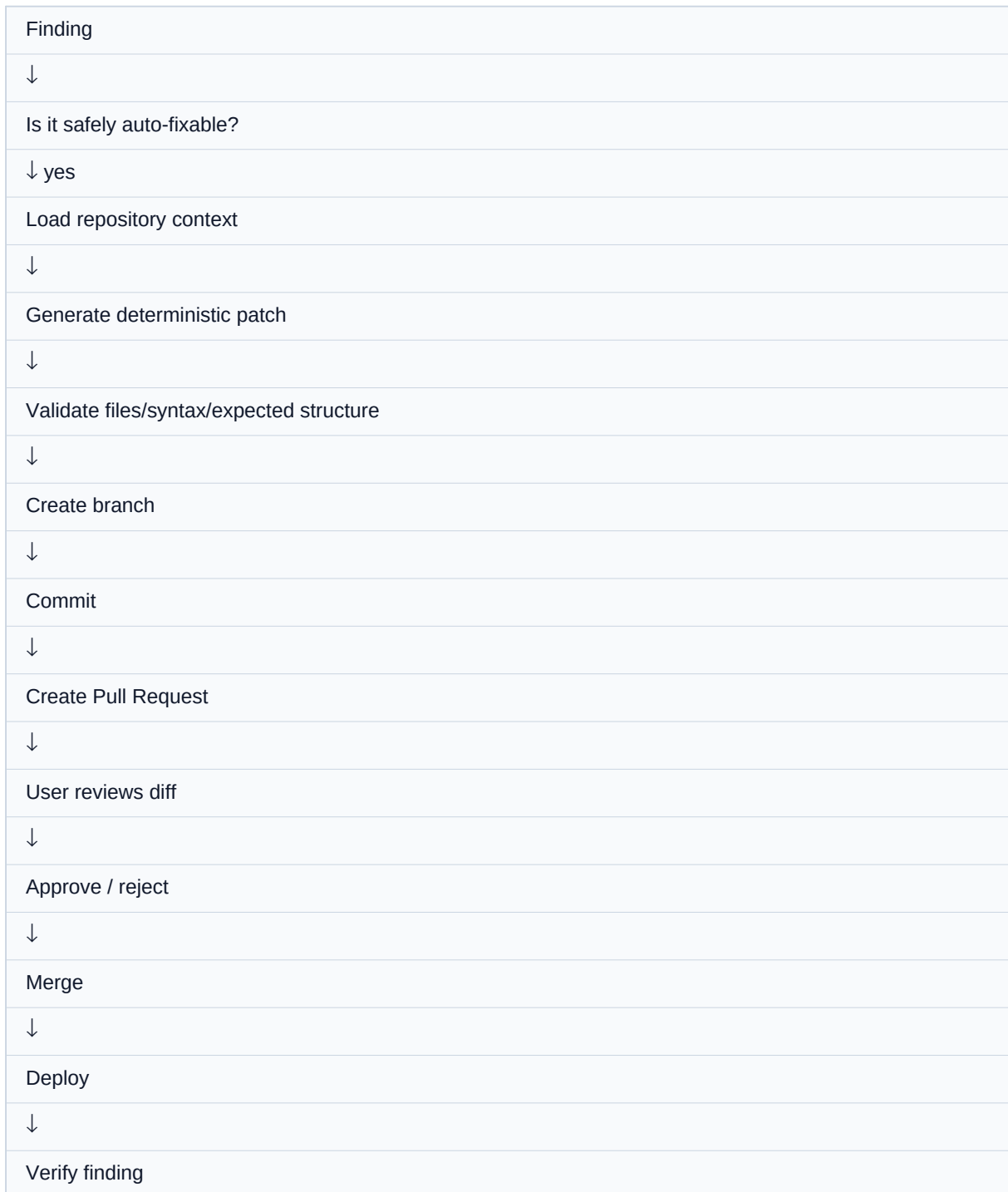
- robots.txt exists and is readable.
- sitemap.xml exists, parses and contains valid URLs.
- HTTP status classification.
- 404 and 5xx detection.
- Redirect chains and loops.
- Missing/duplicate/problematic title tags.
- Missing/duplicate meta descriptions.
- Missing or multiple H1.
- Canonical presence and target validity.
- Noindex/indexability signals.
- Broken internal links.
- Basic image alt-text checks.
- Basic JSON-LD/schema extraction.
- HTTPS/mixed-content signals.
- Mobile/rendered-page checks.

Later rules

- Orphan pages.
- Duplicate/near-duplicate content.
- Internal-link depth.
- Open Graph/social metadata.
- Image dimensions and layout-shift risk.
- Hreflang.
- Pagination/faceted navigation.
- JavaScript-only content.
- Advanced structured-data validation.

12. GitHub Remediation and Approval Gate

This is one of the strongest parts of the concept. Instead of giving a user a checklist, the system can turn safe findings into reviewable pull requests.



Good V1 auto-fixes

- Create/update robots.txt.
- Create/update sitemap configuration.
- Add a known canonical tag.
- Add deterministic metadata when source data is known.
- Add verified JSON-LD boilerplate.

- Create a standard 404 page where the framework structure is known.

Do not automatically merge ambiguous changes, broad content rewrites, routing changes or business-critical application logic. Those remain human-approved proposals.

13. GSC, GA4 and Business Profile

The integrations turn technical SEO into measurable SEO operations.

Integration	Data / capability
GSC	Clicks, impressions, CTR, average position, query/page/device/country dimensions where supported
GSC indexing data	Use supported APIs and available reports; do not assume unsupported fields
GA4	Landing pages, traffic, engagement and configured events/conversions
Business Profile	Business information and performance metrics exposed by the connected APIs
GitHub	Commits, branches, PRs, deployment/change history

Correlation examples

- A deployment occurred → technical crawl changed → organic clicks changed.
- A canonical issue was fixed → affected URLs are monitored for recovery.
- Page performance improved → Lighthouse trend and landing-page engagement are shown together.
- A local landing page changed → local/business-profile metrics can be reviewed.

Important: analytics data should be shown with date ranges, timestamps and clear metric definitions. Avoid presenting historical data as if it were current.

14. Lighthouse and Performance

Use Lighthouse as a lab measurement source. Combine it with field data from Google services when available rather than treating one Lighthouse score as the entire truth about performance.

Metric	Product use
Performance score	Overall lab signal
LCP	Loading experience
INP	Interaction responsiveness
CLS	Visual stability
Accessibility	Common accessibility checks
Best Practices	Common quality/security signals
SEO	Basic automated SEO signals

Execution

Launch isolated browser → open URL → apply desktop/mobile configuration → run Lighthouse → capture JSON report → normalize important metrics → store raw report → compare with previous run → create regression findings

Useful additions

- Compare mobile and desktop.
- Track metric history.
- Show the top causes behind a score.
- Link a performance issue to the exact URL.
- Allow targeted re-runs after a GitHub deployment.

15. Database and Data Model

PostgreSQL should hold application state and normalized metrics. Large raw reports and artifacts can live in S3-compatible storage.

Entity	Important fields
User	id, email, status, created_at
Organization	id, name, plan, settings
Membership	user_id, organization_id, role
Project	id, organization_id, name
Website	id, project_id, url, framework, environment
Integration	provider, account_id, scopes, status, token_ref
Crawl	website_id, status, started_at, completed_at
Page	crawl_id, url, status, title, h1, canonical, indexability
Finding	rule_id, page_id, severity, evidence, status
ChangeProposal	finding_id, files, diff, status
PullRequest	change_id, repo, branch, number, state
LighthouseRun	website_id, url, metrics, report_ref
GSCMetric	website_id, date, dimensions, metrics
GA4Metric	website_id, date, dimensions, metrics
GBPMetric	website_id, date, metrics
AuditLog	actor, action, entity, timestamp, metadata

Keep V1 lean. The broader agency plan contains additional Business, Lead, Competitor, Demo, Outreach, Proposal, Client, Project, Payment and MaintenancePlan entities; those can be added when the agency/CRM layer is built. [filecite turn0file0L447-L463](#)

16. Monitoring and Change Detection

A one-time audit is useful. Continuous monitoring makes the platform sticky and commercially valuable.



Monitor these signals

- New 404/5xx pages.
- robots.txt changes.
- sitemap changes.
- Canonical changes.
- Noindex changes.
- Page-count changes.
- Broken-link spikes.
- Lighthouse regressions.
- GSC click/impression/CTR/position changes.
- GA4 landing-page changes.
- GitHub commits/deployments.
- Local profile changes where supported.

High-value future feature: Deployment-to-SEO Incident Detection

When a GitHub deployment is followed by a sudden increase in 404s, canonical errors or indexing anomalies, show the deployment commit, affected URLs and timeline in one incident view.

17. Optional LLM / AI Architecture

You do **not** need an LLM API key to complete the V1 platform. The core technical SEO product can be implemented entirely with code, rules, browser automation and APIs.

Capability	LLM?	Implementation
HTTP/status checks	No	Code
Robots validation	No	Parser/rules
Sitemap validation	No	XML parser/rules
Canonical checks	No	Rules
Metadata checks	No	Rules
Lighthouse	No	Lighthouse
GSC/GA4 sync	No	Google APIs
GitHub PR	No	GitHub API
Issue explanation	Optional	Templates first
Keyword/content strategy	Useful	LLM + search evidence
Content brief	Useful	LLM + retrieved data
Complex code generation	Useful	LLM + tests + approval
SEO Copilot	Useful	LLM + project RAG

Recommended AI flow

Deterministic SEO engine
↓
Structured findings + GSC/GA4/GBP/GitHub context
↓
Optional LLM
↓
Recommendation / explanation / generation
↓
Validation
↓
Human approval
↓
GitHub PR / content proposal

AI should reason over evidence, not invent the measurements. Add paid API usage only after the product proves where model quality creates real value. The supplied operating plan makes the same recommendation. ■filecite■turn0file0■L502-L506■

18. Security, Testing and Production

Security

- Never store secrets in source control.
- Use least-privilege OAuth scopes.
- Encrypt sensitive tokens where stored.
- Validate URLs, files and webhook payloads.
- Rate-limit public APIs.
- Separate development/staging/production secrets.
- Verify GitHub webhook signatures.
- Log administrative and automated actions.
- Do not execute arbitrary customer repository code on the API server.
- Use isolated containers/sandboxes for future repository build/test execution.
- Respect robots directives, API limits and access restrictions.

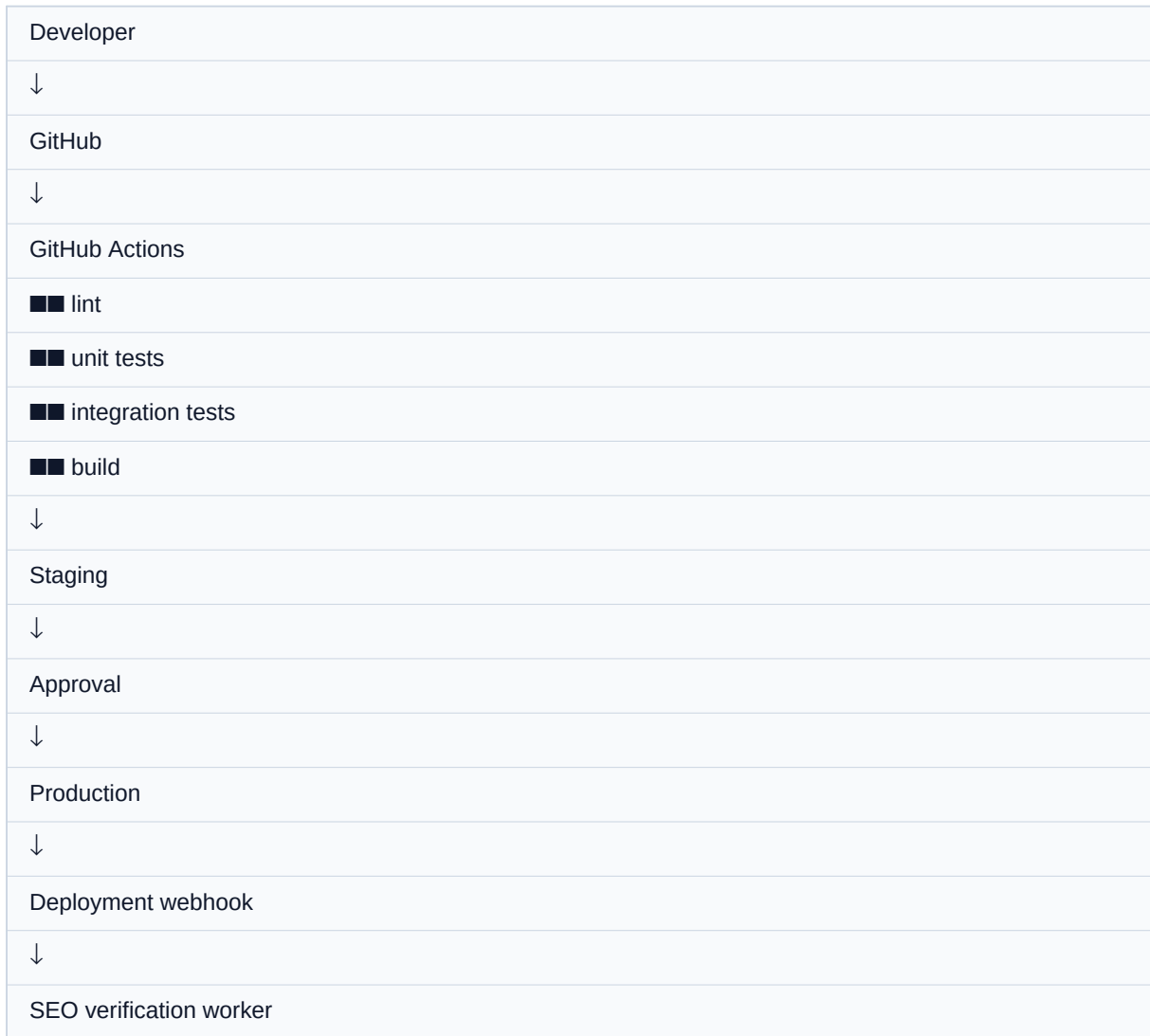
Testing

- Unit tests for every SEO rule.
- Fixture websites for crawler tests.
- Mock GSC/GA4/GitHub integrations.
- Integration tests for OAuth and PR creation.
- End-to-end test: crawl → finding → proposal → PR → verification.
- Regression tests for known website frameworks.
- Security tests for token access and authorization.
- Failure tests for expired OAuth tokens and unavailable external APIs.

Production guardrail

A failed validation must block the PR/merge path. A risky or ambiguous fix should become a recommendation rather than an automatic code change.

19. Deployment and DevOps



Practical initial topology

Component	Initial hosting direction
Next.js	Vercel
Node API	Render / Railway / AWS
PostgreSQL	Managed PostgreSQL
Redis	Managed Redis
Python crawler	Worker container
Object storage	S3-compatible
CI/CD	GitHub Actions
Monitoring	Application logs + error tracking

Use Docker Compose locally to run the API, PostgreSQL, Redis and worker stack consistently. Move to separate managed services only when real usage requires it.

20. Final Product Modules

Module	Final capability
1. Project Management	Organizations, websites, users, roles
2. Integrations	GitHub, GSC, GA4, Business Profile
3. Website Intelligence	Crawler, browser rendering, page inventory
4. Technical SEO	Rules, findings, evidence, severity
5. Performance	Lighthouse, CWV, regression history
6. Remediation	Change proposals, diffs, GitHub PRs
7. Approval	Human review and controlled merge
8. Verification	Post-deployment crawl and fix validation
9. Analytics	GSC + GA4 + local metrics
10. Monitoring	Scheduled crawls, syncs and alerts
11. Reporting	Technical, executive and monthly reports
12. Content SEO	Keywords, content gaps, briefs
13. Local SEO	GBP, location pages, local signals
14. AI	Copilot, recommendations and generation
15. Agency Mode	Multi-client workspaces, white-label and teams
16. Billing	Plans, usage limits and subscriptions

The final platform therefore becomes much more than an SEO checker. It becomes an **SEO operations system** with a controlled path from discovery to implementation and measurement.

21. Implementation Roadmap

Phase	Build	Exit criterion
0	Repo, environments, auth, database	Full local stack runs
1	Projects + website onboarding	Project can be created
2	GitHub integration	Repository safely accessible
3	Crawler + page inventory	Reliable crawl stored
4	Technical rules	Findings reproducible
5	Robots + sitemap automation	Create/update + validation works
6	Lighthouse	Reports stored/displayed
7	GSC + GA4 + GBP	Data visible
8	Change proposals + PR	Finding can become PR
9	Approval + merge + verification	Closed loop works
10	Monitoring	Regressions detected
11	Reports	Client-ready output
12	AI layer	Optional AI recommendations
13	Content/local SEO	Broader SEO
14	SaaS scale	Teams, plans, billing, limits

Recommended immediate priority

Finish the loop you already have. Make GitHub → audit → PR → approval → merge → deployment verification rock-solid before adding many new modules.

The uploaded local-business plan similarly recommends a staged roadmap and warns against spending months building automation before validating the business. ■filecite■turn0file0■L528-L555■

22. Final Production Checklist

Area	Acceptance criterion	Done
Auth	Secure authentication + authorization	■
Projects	Website project lifecycle works	■
GitHub	OAuth, repo selection and permissions	■
Crawler	Stable crawl + dedupe + retries	■
Robots	Read/validate/create/update	■
Sitemap	Read/validate/create/update	■
Metadata	Title/description/H1 checks	■
Canonical	Extraction + target validation	■
Links	Broken/internal-link checks	■
Schema	Extraction + validation baseline	■
Images	Alt/size/basic checks	■
Lighthouse	Run + store + compare	■
GSC	Connect + sync + dashboard	■
GA4	Connect + sync + dashboard	■
GBP	Connect where supported	■
Findings	Severity + evidence + status	■
Change proposal	Before/after diff	■
GitHub PR	Branch/commit/PR workflow	■
Approval	No risky change without approval	■
Verification	Post-deploy check closes findings	■
Monitoring	Scheduled sync/crawl + alerts	■
Security	Secrets, scopes, logs, limits	■
Backups	Recovery tested	■
Observability	Logs + errors + health checks	■
Reports	Exportable reports	■
Documentation	Setup/runbook/API docs	■

Final product principle

Build the deterministic foundation first. Every audit result should be evidence-based. Every automated code change should be reviewable. Every deployment should be verifiable. Then expand into content SEO, local SEO, monitoring and AI-assisted automation.

The broader business blueprint describes the same long-term progression: website delivery → maintenance/SEO → analytics/reporting → additional features and recurring services. ■filecite■turn0file0■L239-L275■